

Appunti del corso di **Linguaggi**

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis **Matteo Drago**

Indice

1	Procedure	1
1.1	Regole di scope	2
1.1.1	Scope statico	3
1.1.2	Scope dinamico	4
1.2	Visibilità delle dichiarazioni	4

Procedure

L'uso delle procedure permette l'**astrazione del controllo**.

Nota

Astrarre significa nascondere dettagli facendo emergere caratteristiche comuni.

L'astrazione chiede di identificare le proposizioni importanti che si vuole descrivere al fine di concentrarsi che si vuole descrivere al fine di concentrarsi sulle questioni rilevanti e ignorare quelle non importanti.

Quando si parla di astrazione del controllo ci sono, in realtà, due categorie di sottoprogrammi con nome che possiamo definire:

- ◇ **procedure** (no output)
- ◇ **funzioni** (restituisce output)

Parte importante del processo di astrazione consiste nel dare al raggruppamento un nome.

Il **nome** permette di riferirsi all'astrazione senza conoscerne i dettagli, una sola le caratteristiche di interesse che ne hanno determinato il raggruppamento.

I **parametri** permettono l'astrazione di tante computazioni simili che cambiano di tante computazioni simili che cambiano solo per alcuni input a cui vengono applicate e che vengono abbreviate con un singolo nome.

Il **corpo** è ciò che viene valutato/eleborato/eseguito ogni volta che il nome viene richiamato.

Una procedura viene definita in tre fasi:

1. **specifica**: descrizione dell'interfaccia della procedura/funzione

```
1  double P(int x) {  
2  
3      }  
4
```

2. **definizione**: descrizione del corpo della procedura/funzione

```
1  {  
2      double y;  
3      //corpo procedura P
```

```

4      ...
5      (return y);
6  }
7

```

3. **uso**: chiamata della procedura/funzione per eseguire il codice

```

1      int n;
2      double z;
3      z = P(n); //chiamata della procedura
4

```

Consideriamo il seguente codice:

```

1      int foo(int n) {
2          int tmp = a;
3
4          if(tmp == 0)
5              return n;
6          else
7              return n + 1;
8      }

```

Listing 1.1: programma di esempio

In questo caso la variabile *a* è un identificatore non locale, cioè un identificatore libero per cui di deve trovare il significato nel contesto.

Il problema è che nel contesto del chiamante potrebbero esserci più definizioni di *a* e non è chiaro quale sia quella a cui si riferisce la procedura.

Di conseguenza, la regola di visibilità delle dichiarazioni non è più sufficiente per risolvere i riferimenti quando si usano le procedure se la procedura ha un ambiente non locale.

Definizione: Ambiente di una procedura

L'**ambiente di riferimento** di una procedura è l'insieme di tutti i nomi che sono visibili al comando.

L'ambiente di riferimento di una procedura è determinato da varie tipologie di regole:

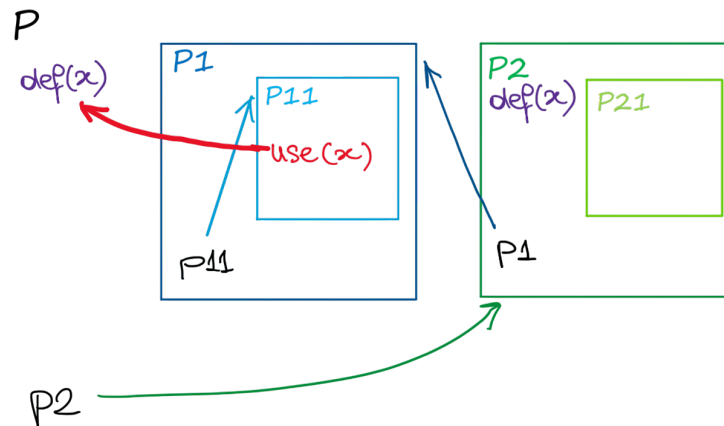
- ◇ **regole di scope** (statico o dinamico)
- ◇ **regole di visibilità**: determinano quali dichiarazioni sono visibili in un certo punto del programma
- ◇ **regole di binding** (shallow o deep): determinano a quale dichiarazione si riferisce un certo identificatore

1.1 Regole di scope

Le regole di scope possono essere distinte in due tipologie:

- ▷ **scope statico**: prevale il contesto di definizione.
L'ambiente di riferimento è quello attivo al momento della definizione della procedura.
- ▷ **scope dinamico**: prevale il contesto di esecuzione.
L'ambiente di riferimento è quello attivo al momento della chiamata della procedura.

1.1.1 Scope statico



Con lo scope statico il riferimento per x è P , ovvero si risale la catena di annidamento dei blocchi (**catena statica**), nota a tempo di compilazione.

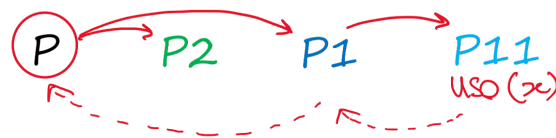


Figura 1.1: catena di definizione

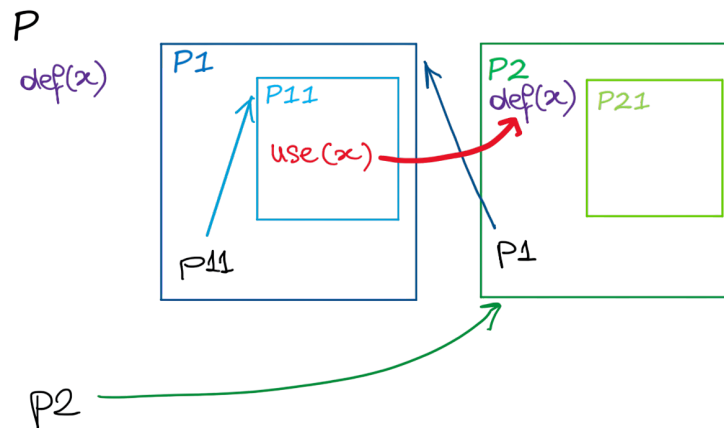
```

1  {
2      int x = 0;
3
4      void pippo(int n) {
5          x = n + 1;
6      }
7
8      pippo(3);
9      write(x); //stampa in ogni caso 4
10
11     {
12         int x = 0;
13         pippo(3);
14         write(x);
15         //stampa 0 perche' pippo(3) va a modifica il primo valore
16         //di x, ma la write si riferisce all'ultima istanza (x = 0)
17     }
18
19     write(x); //stampa in ogni caso 4
20 }

```

Listing 1.2: codice esempio con scope statico

1.1.2 Scope dinamico



Con lo scope dinamico il riferimento per x è P2. ovvero si risale la catena di chiamate dei blocchi (**catena dinamica**), nota a tempo di compilazione.



Figura 1.2: catena di definizione

```

1  {
2      int x = 0;
3
4      void pippo(int n) {
5          x = n + 1;
6      }
7
8      pippo(3);
9      write(x); //stampa in ogni caso 4
10
11     {
12         int x = 0;
13         pippo(3);
14         write(x);
15         //stampa 4 perche' pippo(3) va a modifica il secondo valore
16         //di x
17     }
18
19     write(x); //stampa in ogni caso 4
20 }

```

Listing 1.3: codice esempio con scope dinamico

1.2 Visibilità delle dichiarazioni

Una dichiarazione può essere visibile:

▷ a partire dalla dichiarazione e fino alla fine del blocco

```
1      {  
2          a := ...  
3          .  
4          .  
5          .  
6          int i := int //visibile da qui in giu'  
7      }  
8
```

▷ all'interno del blocco visibile sempre